

Sensor Systems

Laboratory 01
ARM – Cortex microcontrollers

16.09.2026 I Federica Villa, Arianna De Vecchi

Contents

1. ARM-Cortex cores
2. Exceptions, Interrupts, and Faults
3. STM32 microcontroller
4. NUCLEO board
5. POLIMI expansion board

1. ARM-Cortex cores

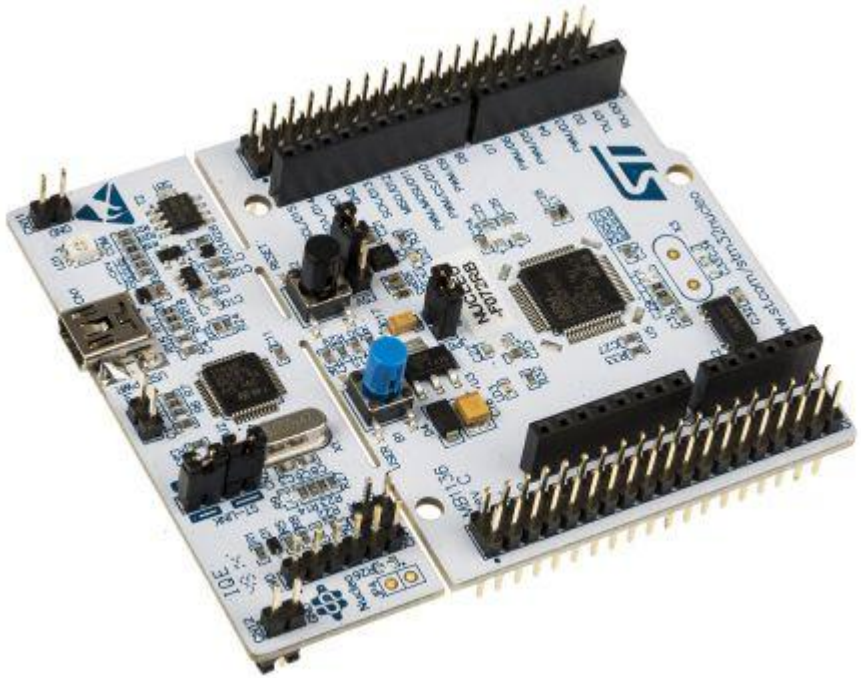
01

Embedded system definition

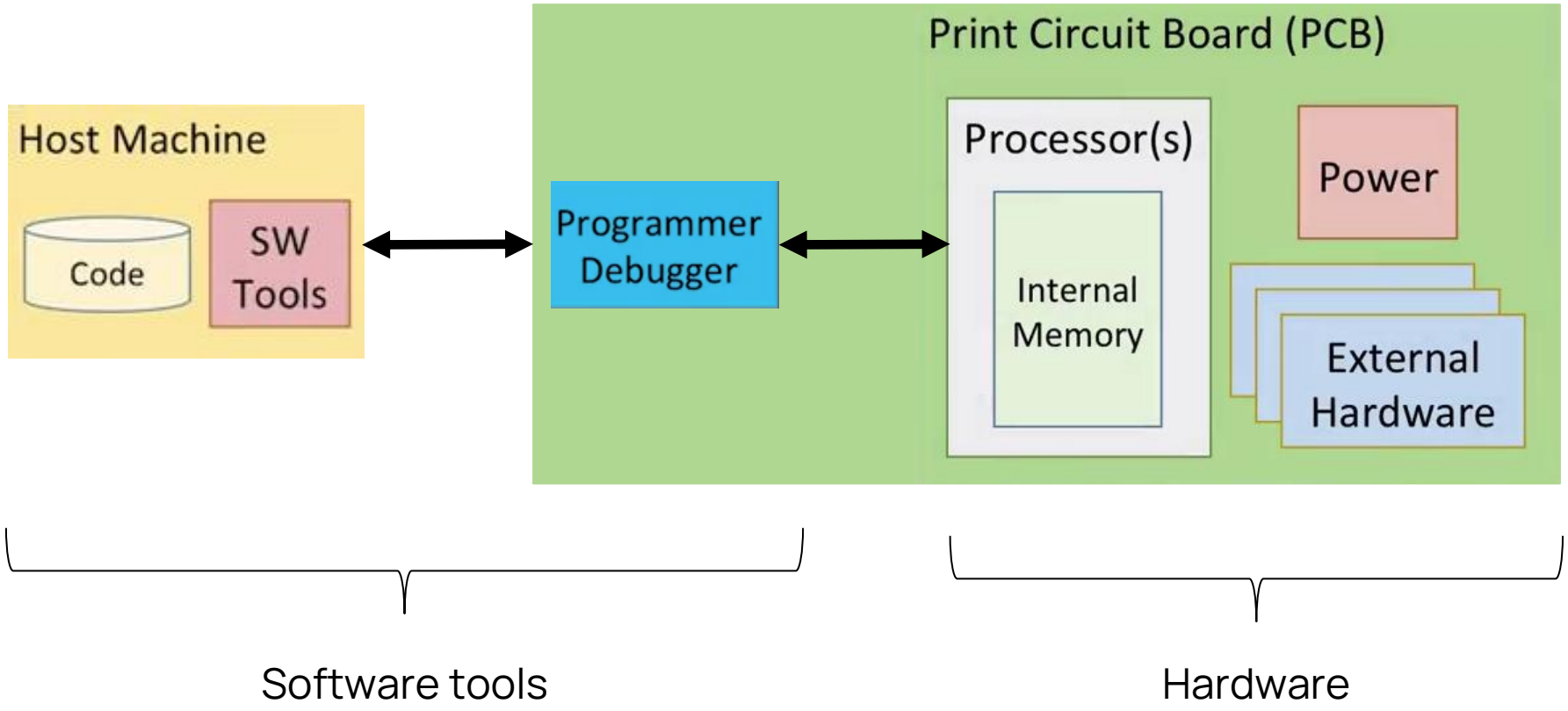
An **embedded system** is a microprocessor-based computer hardware system with software that is designed to perform a dedicated function, either as an independent system or as a part of a large system.

Limited resources:

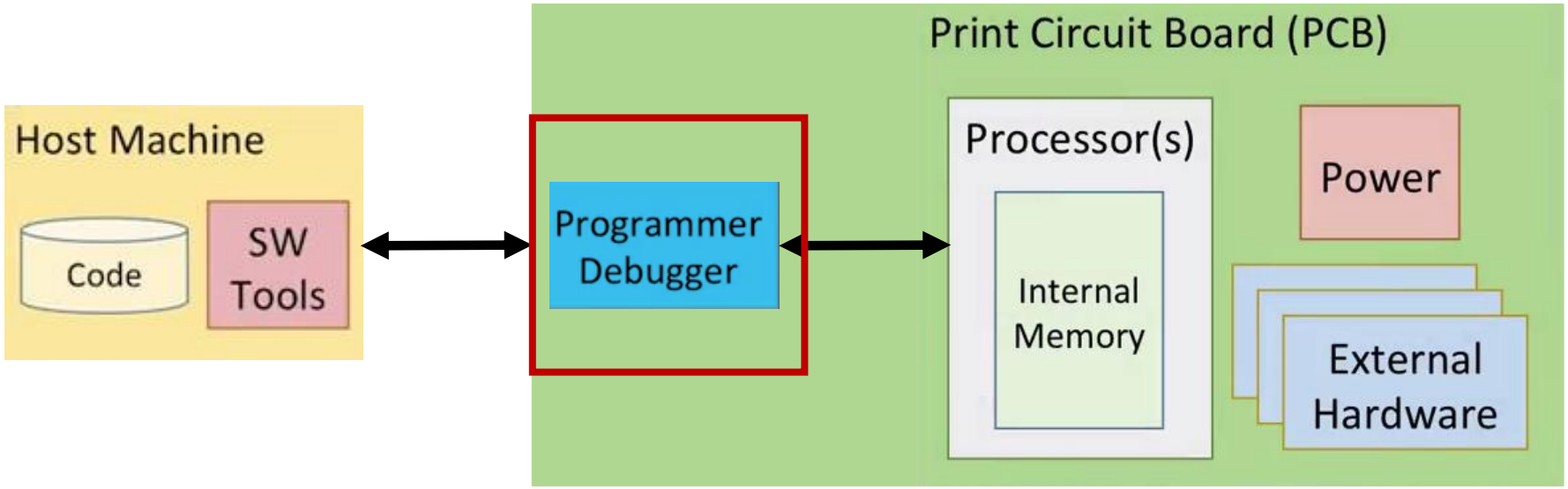
- Processing
- Memory
- Peripherals



Development platform



Development platform

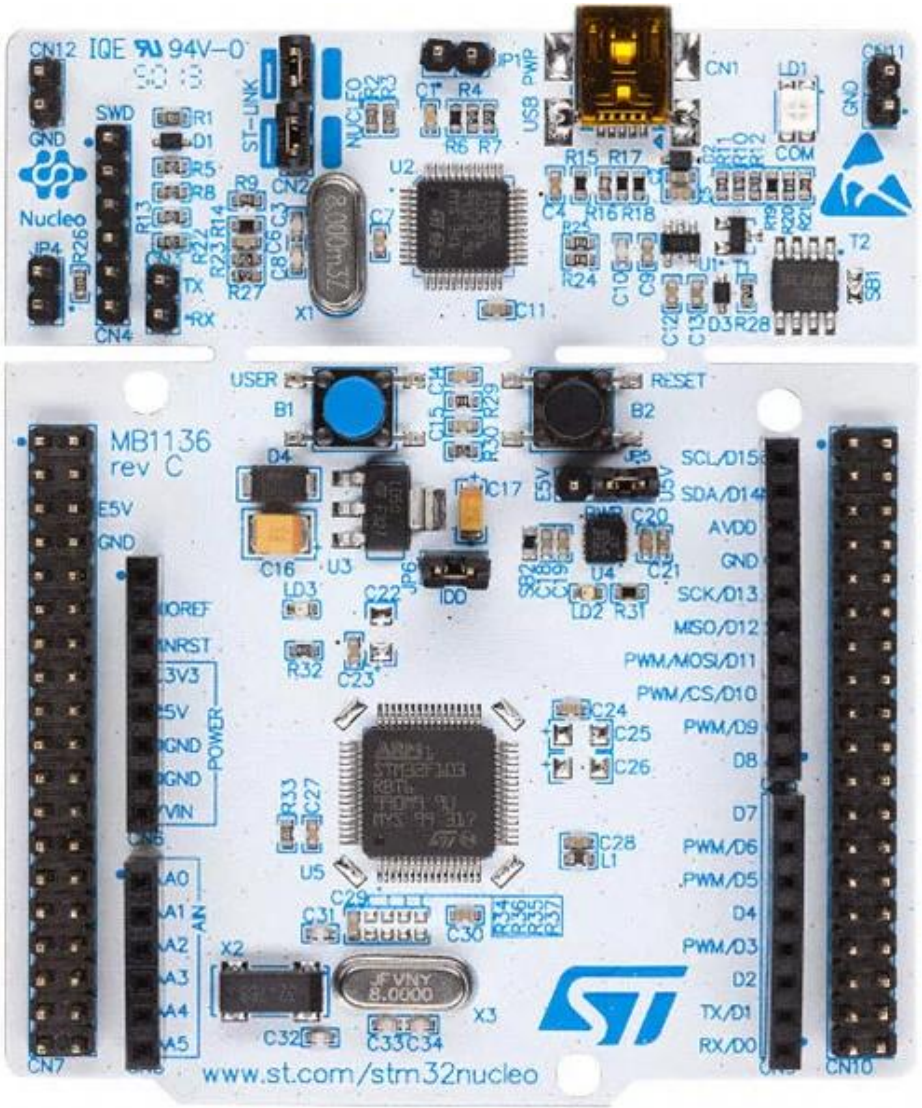


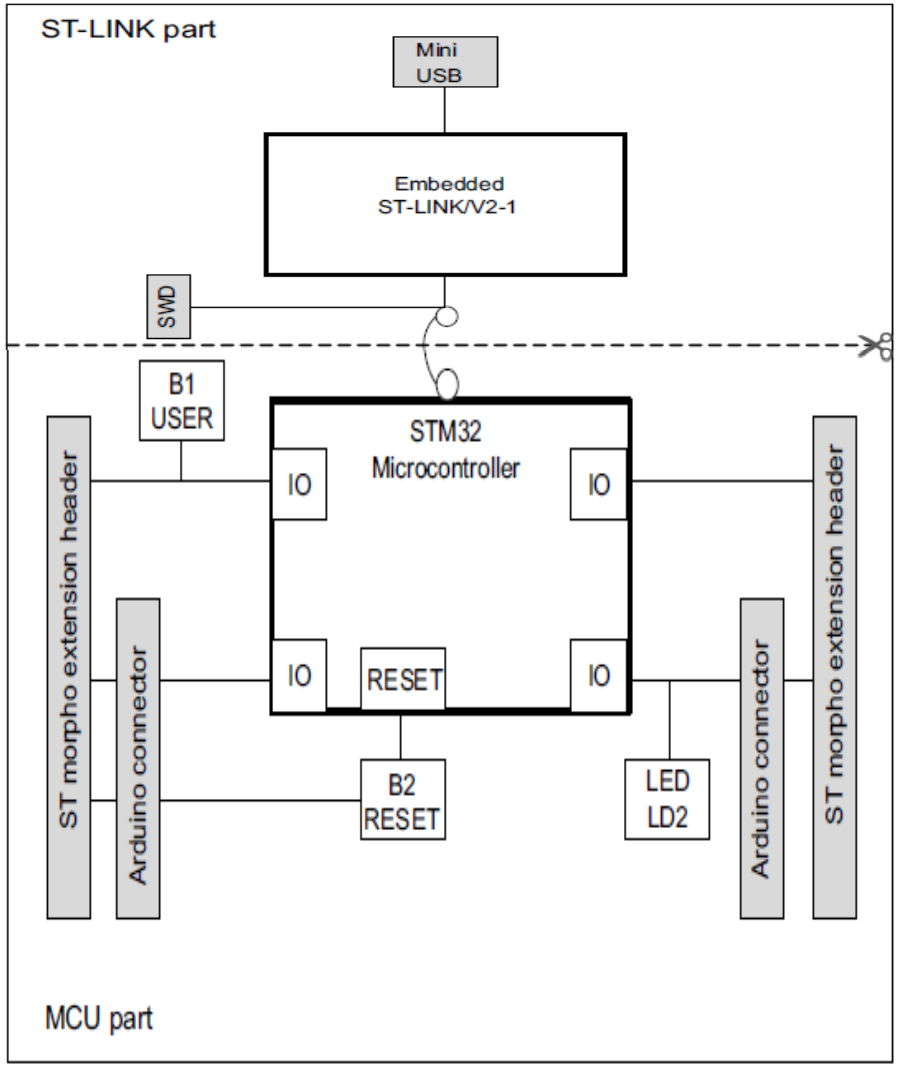
The programmer and debugger interface **facilitates bi-directional communication** between the host (PC) and the microcontroller.

It allows the user to perform **real-time debugging** directly on the microcontroller by:

- Pausing execution.
- Inspecting and modifying memory and registers.
- Setting breakpoints to halt execution at specific code locations.
- Single-stepping through instructions to observe behavior.

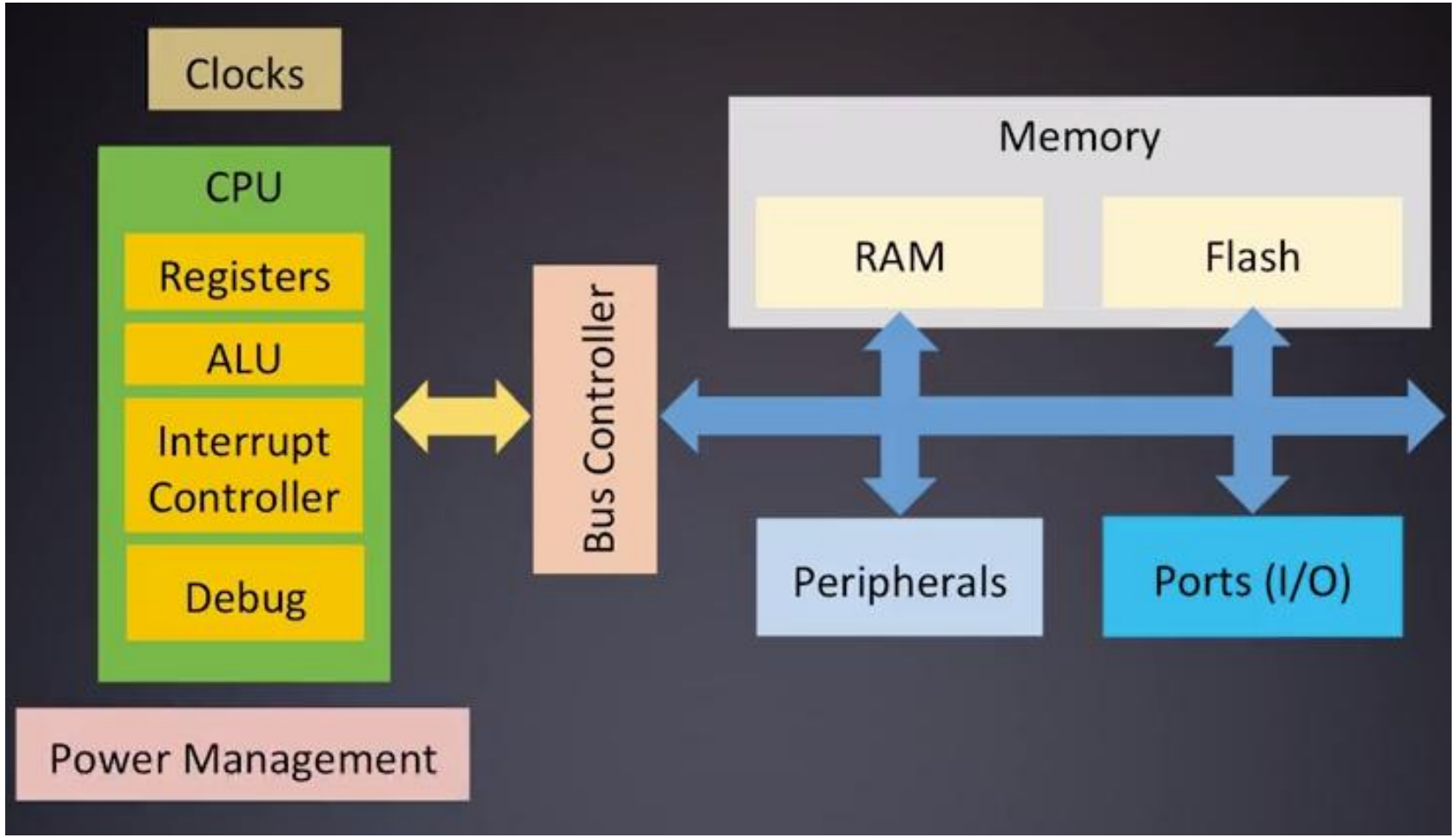
Example of a development platform





Microcontroller definition

A microcontroller is a microprocessor (CPU)
with added functionality, such as memory and peripheral hardware.



CPU and memories and peripherals

CPU (Central Processing Unit)

= core of the microcontroller that executes instructions and controls system operations:

- Executes program instructions
- Performs calculations (ALU)
- Controls peripherals and data flow
- Manages interrupts

Memories

- **Flash**: non-volatile, stores the program
- **RAM**: volatile, stores variables and temporary data

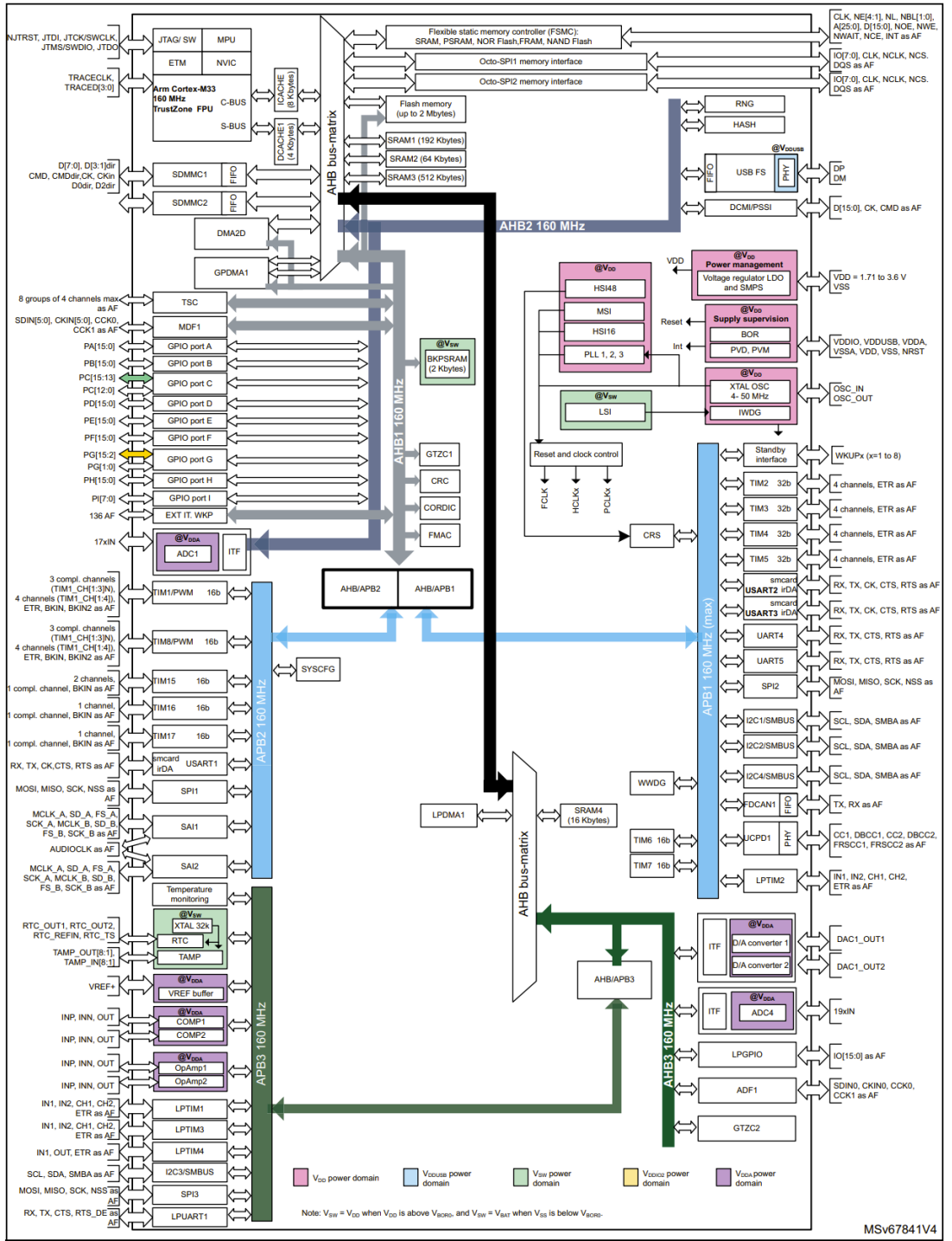
Peripherals

= **hardware modules** inside a MCU that perform a specific function independently of the CPU (CPU and peripherals work in parallel).

Peripherals are connected to CPU via internal **buses**.

Peripherals interacts with the external world or handle internal tasks.

- **Communication:** I2C, SPI, USART…
- **Timing:** clocks, PWM…
- **Analog** signal processing: ADC, DAC…
- **General Purpose Input/Output** (GPIO)
- **Power** management



Why ARM?



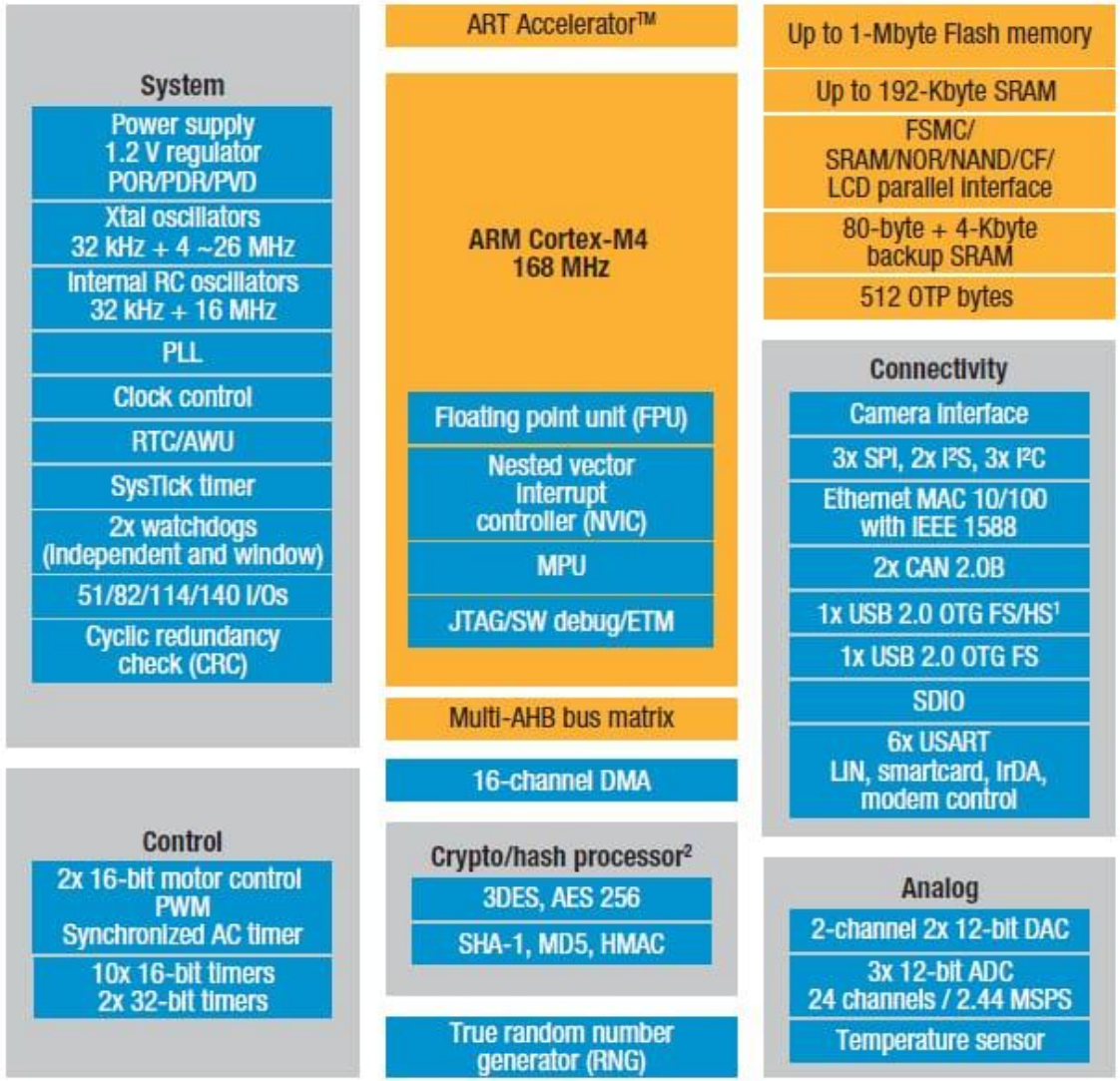
ARM®

Advanced RISC Machine

Arm Holdings provides the I.P. for the core processor

ARM – Cortex → 32 bit registers

STM32F4 block diagram



RISC vs CISC

ARM = Advanced RISC Machine

RISC: Reduced Instruction Set Computer

Based on very simple instructions that execute within a single cycle, at high clock speed.

VS

CISC: Complex Instruction Set Computer

Based on more complex instructions to complete a task in as few lines as possible.

Example: how to execute 1800 + 1801

CISC

ADD 1800, 1801



Memory locations

RISC

Load X, 1800
Load Y, 1801
ADD X, Y
Store 1800, X

RISC vs CISC

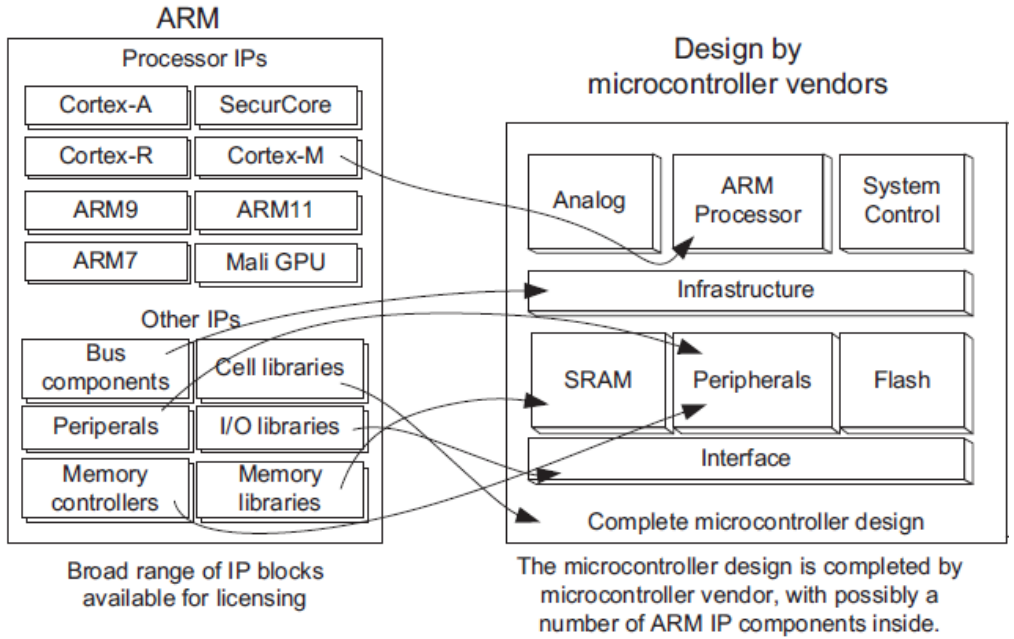
CISC

- Little work from the compiler to pass from high-level language into assembly (and little memory needed)
- Short codes
- More clock cycles
- More difficult pipeline
- More transistors used to store instructions

RISC

- Long codes and more RAM needed (and it must also be fast)
- More general purpose hardware
- One cycle per instruction (or even less)
- Easier pipeline
- More transistors used for register memories

ARM IP model and families



Business model:

Intellectual

Property

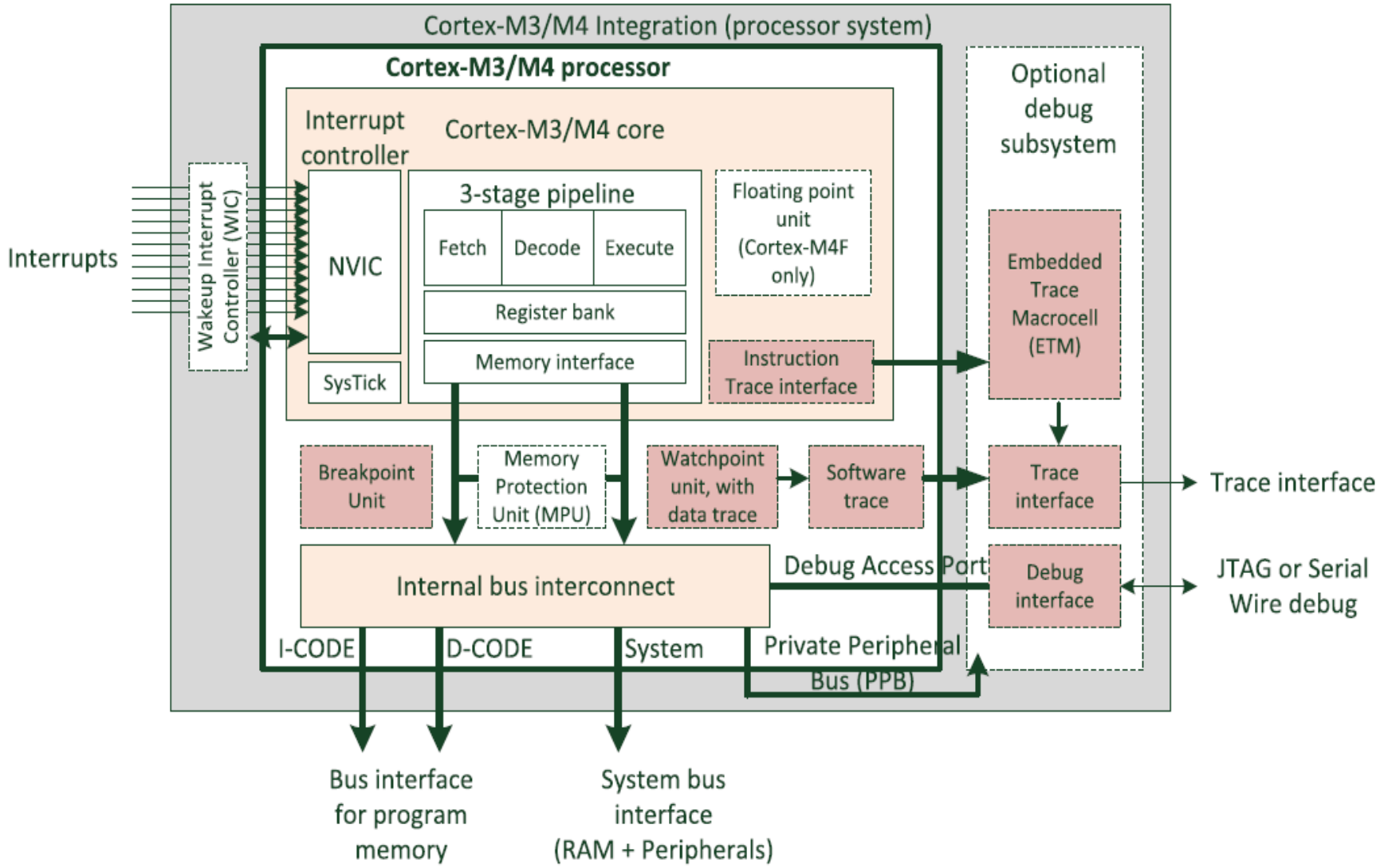
Licensing



- A: application (high-end)
- R: real time (more focus on timing performance)
- M: more wide-range and mainstream

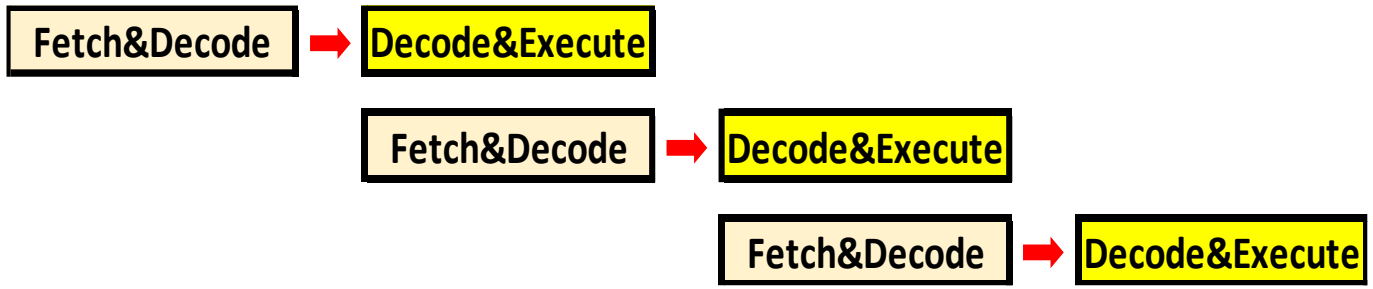


ARM-Cortex M3 and M4 architectures

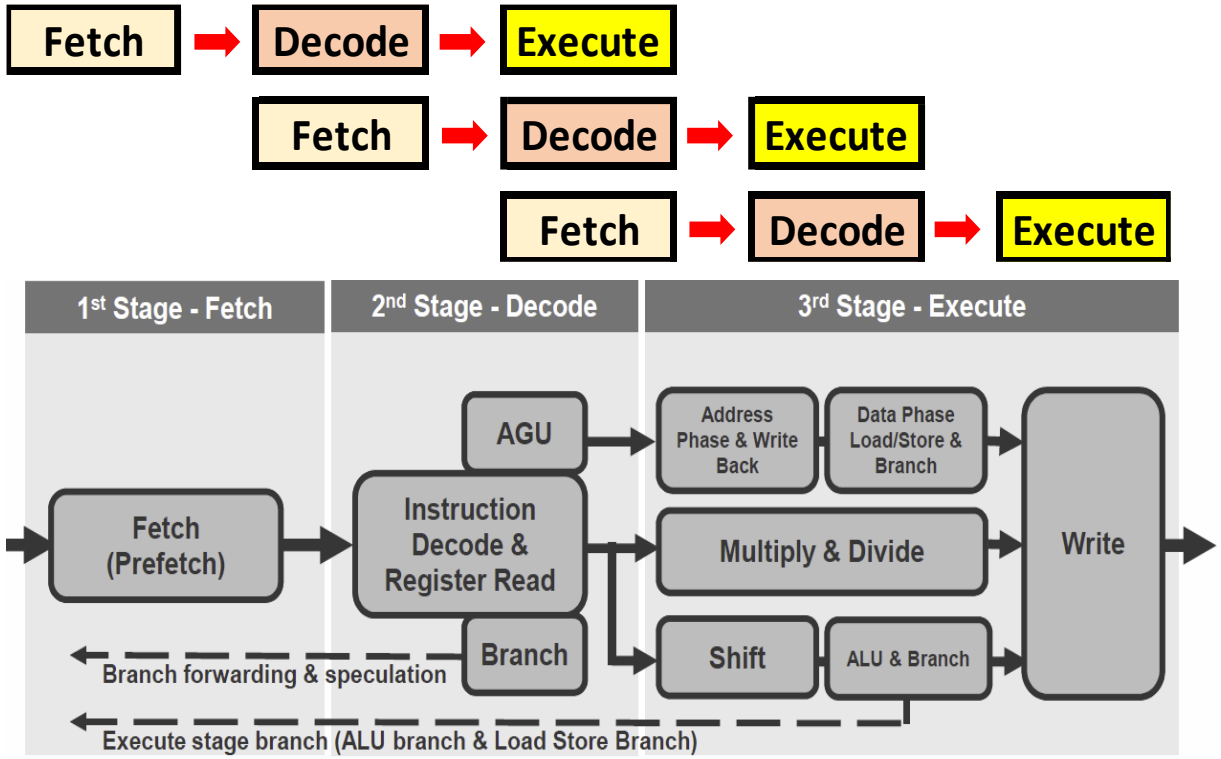


ARM-Cortex pipelining

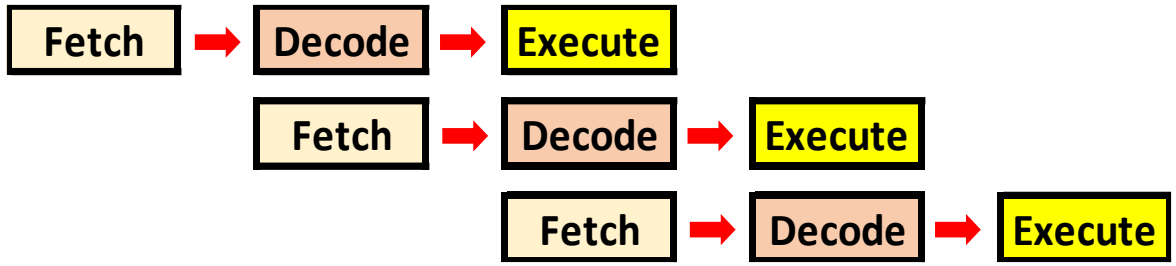
2-level (M0+)



3-level (M3/4)



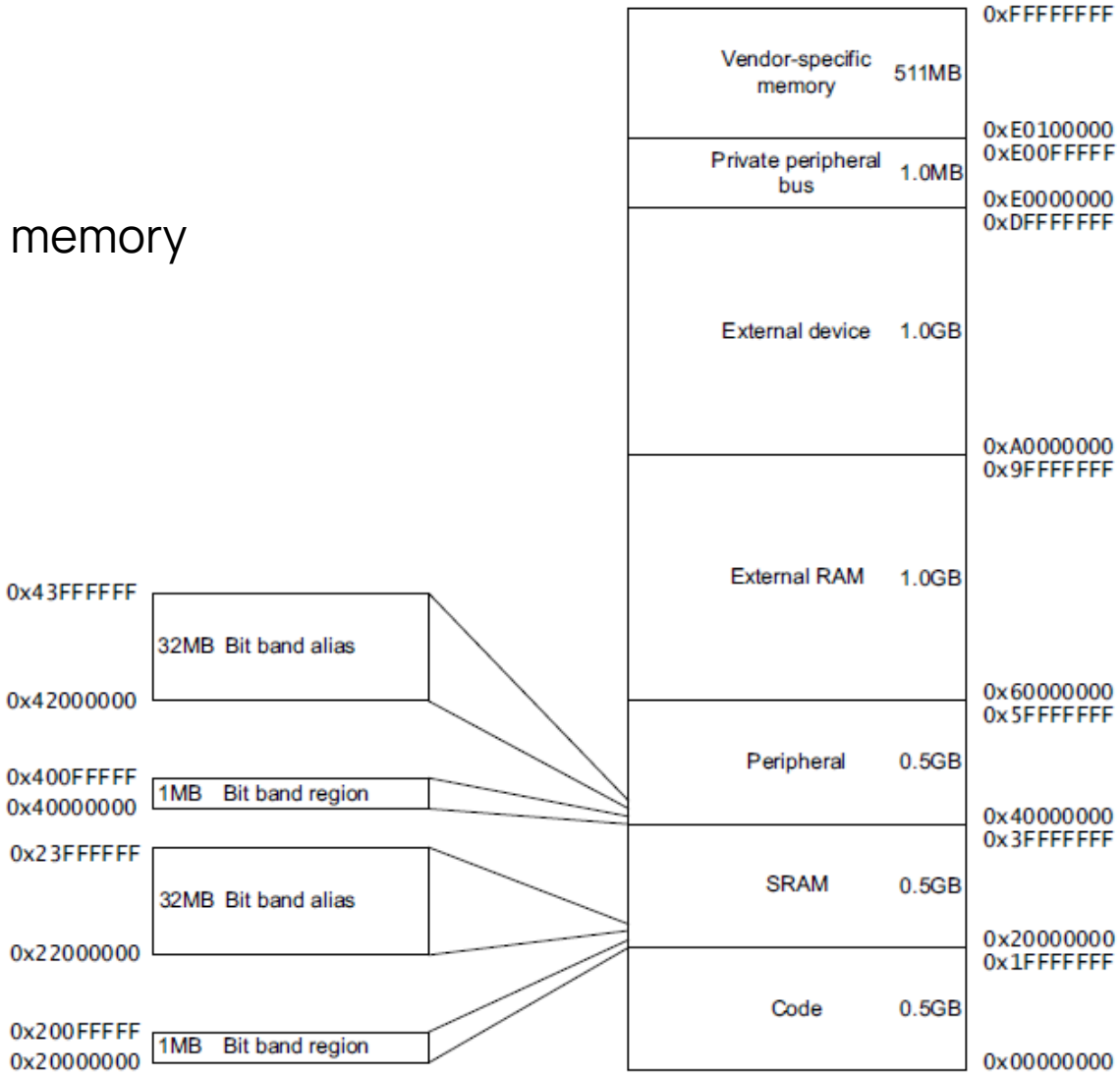
ARM-Cortex pipelining



- **Fetch**: the processor fetches (picks) the instruction from memory (usually from flash or RAM) and loads it into the instruction register.
- **Decode**: the fetched instruction is decoded to determine what kind of operation needs to be performed.
- **Execute**: the instruction is executed, meaning the actual operation (e.g., an arithmetic operation, data transfer, or branching) is carried out.

ARM-Cortex M4 memory map

4 GB addressable memory



ARM-Cortex M4 memory map

Code (0x00000000 - 0x1FFFFFFF)

Where the program code is stored. It is a Flash memory (non-volatile), meaning the contents are preserved when power is off.

SRAM (0x20000000 - 0x3FFFFFFF)

Static RAM (SRAM), used for storing variables, data, and stack information during the execution of a program. It is volatile.
No need to be constantly refreshed, unlike DRAM, so:

- Faster
- Easier to use
- Less consumption
- More expensive

Peripheral (0x40000000 - 0x5FFFFFFF)

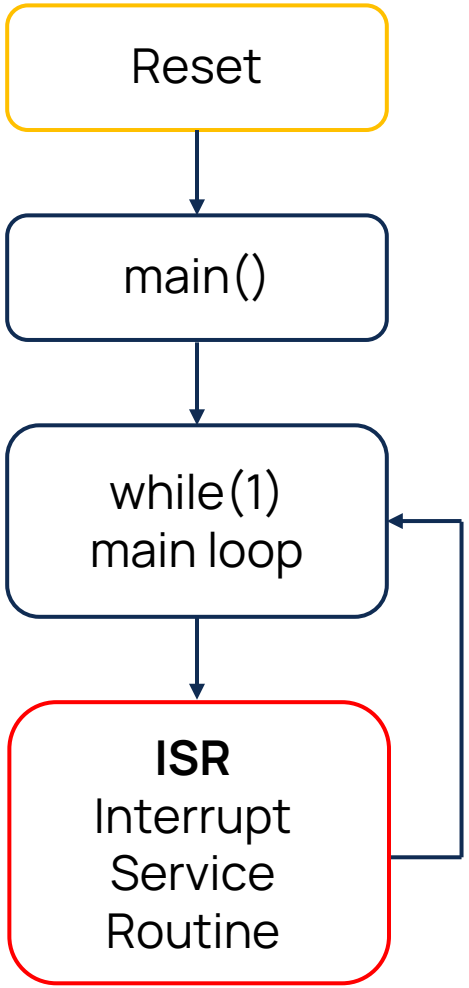
Reserved for the microcontroller's peripherals, such as timers, ADCs (analog-to-digital converters), DACs (digital-to-analog converters), and communication interfaces like UART, SPI, and I2C.

2. Exceptions, Interrupts, and Faults

02

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

Code Flow in ARM Microcontrollers

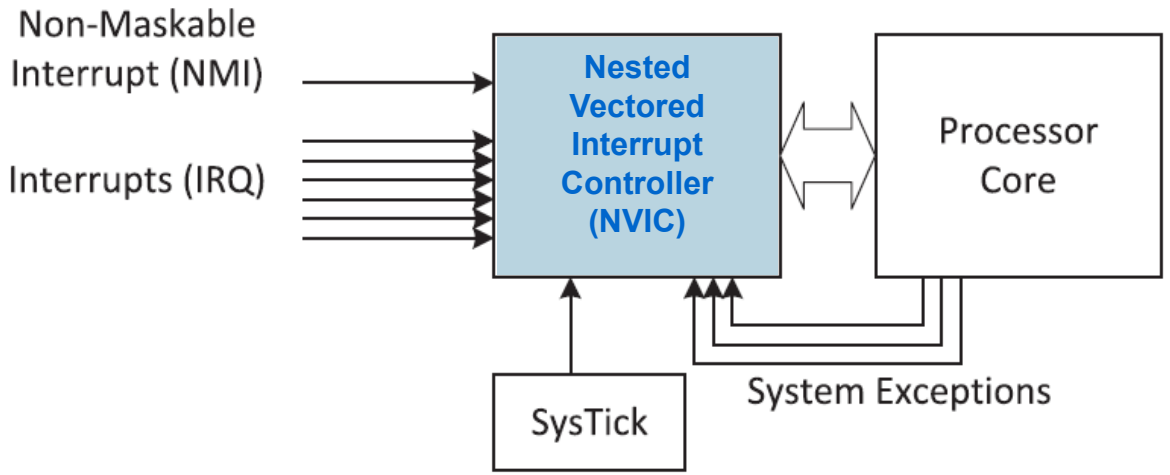


- Main loop runs **continuously** in the CPU
- Interrupts handle **asynchronous events** from the peripherals
- CPU temporarily **pauses execution** of the main and manages the interrupt in the Interrupt Service Routine (ISR)
- After ISR → returns to main loop

```
while(1) {
    instruction main 1
    instruction main 2
    instruction main 3
    instruction main 4
    ...
}
```

```
interrupt {
    instruction int 1
    instruction int 2
    ...
}
```

ARM-Cortex M4 exceptions and interrupts



The ARM-cortex M4 features both exceptions and interrupts.
Both are handled by the Nested Vectored Interrupt Controller (NVIC).

ARM-Cortex M4 exceptions and interrupts

Exception sources:

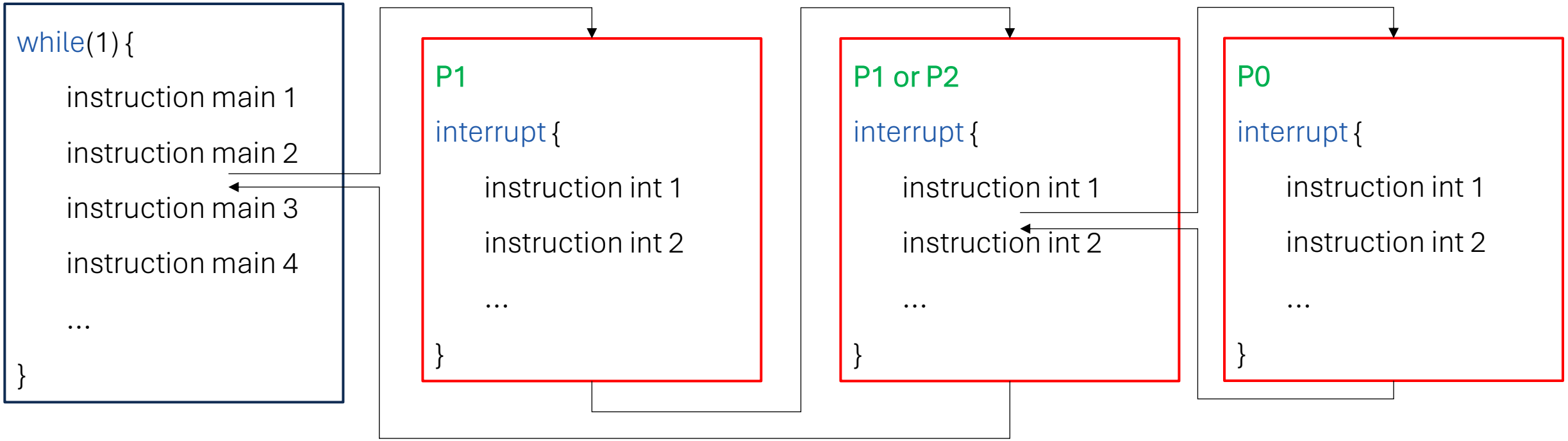
- **Reset** → Processor startup or restart; enters privileged Thread mode
- **NMI** → Non-maskable interrupt with the highest priority after Reset.
- **HardFault** → Severe fault occurring during exception processing.
- **MemManage** → Memory protection violation during instruction or data access.
- **BusFault** → Error during an instruction or data transfer on the memory bus.
- **UsageFault** → Instruction execution error, e.g. undefined instruction, unaligned access, division by zero.
- **SVCall** → Software-generated exception used to request OS/kernel services.
- **PendSV** → System-level service request, mainly used for context switching in an RTOS.
- **SysTick** → System timer exception, commonly used to generate a periodic system tick.
- **IRQ (Interrupt Request)** → Interrupt request generated by a peripheral.

Interrupts

Interrupts can be generated by:

- MCU peripherals (Timers, ADC…)
- External events (EXTI) → changes in GPIO pins

Nested Interrupt Vector (**NVIC**) manages the interrupt priorities



ARM-Cortex M4 interrupt vector

Exception priorities:

lower priority number = higher priority

Exception number	IRQ number	Type	Priority	Vector address or offset	Activation
1		Reset	-3 the highest	0x00000004	Asynchronous
2	-14	NMI	-2	0x00000008	Asynchronous
3	-13	HardFault	-1	0x0000000C	
4	-12	MemManage	configurable	0x00000010	Synchronous
5	-11	BusFault	configurable	0x00000014	Synch/Asynch
6	-10	UsageFault	configurable	0x00000018	Synchronous
7-10	…	Reserved			
11	-5	SVCall	configurable	0x0000002C	Synchronous
12-13	…	Reserved			
14	-2	PendSV	configurable	0x00000038	Asynchronous
15	-1	SysTick	configurable	0x0000003C	Asynchronous
16	0	Interrupt IRQ_0	configurable	0x00000040	Asynchronous
17	1	IRQ_1	configurable	0x00000040	Asynchronous
…	…	$IRQ_{…}$	configurable	…	Asynchronous
…m…255	…m…239	IRQ_{m-16}	configurable	0x00000040+ +(m-16)·4	Asynchronous

Vector Table:

stores the starting addresses of all exception handlers.

Exception number	IRQ number	Offset	Vector
16+n	N		IRQn
.	.		.
17	1		IRQ1
16	0	0x00000040	IRQ0
15	-1	0x0000003C	SysTick
14	-2	0x00000038	PendSV
13			reserved
12			reserved for Debug
11	-5	0x0000002C	SVCall
10			
9			reserved
8			
7			
6	-10	0x00000018	UsageFault
5	-11	0x00000014	BusFault
4	-12	0x00000010	MemManagement fault
3	-13	0x0000000C	HardFault
2	-14	0x00000008	NMI
1		0x00000004	Reset
		0x00000000	Initial SP value

2. STM32 microcontroller

02

STM32 family



STM32F4

- High performance
- Up to 180 MHz clock

STM32F4xx

STM32F4xx:

- Cortex M4 core
- Different maximum clock speeds
- Different flash sizes
- Different RAM sizes

- ART Accelerator™ enabling 0 wait state executing from internal Flash
- Up to 2x USB2.0 OTG FS/HS (except for access lines)
- SDIO
- USART, SPI, I²C
- I²S + audio PLL
- 16 and 32-bit timers
- Up to 3x 12-bit ADC (0.41 µs)
- Up to 2x 12-bit DAC
- External memory controller
- Low voltage 1.71 to 3.6 V

STM32 F4	FCPU (MHz)	Flash (bytes)	RAM (KB)	Ethernet I/F IEEE 1588 / 2x CAN	Camera I/F	SDRAM I/F / Dual Quad-SPI	SAI3 I/F / SPDIF RX	Chrom-ART Grphic Accelerator™	TFT LCD controller	MPI DSI
Advanced lines										
STM32F469[2]	180	512 K to 2 M	384	• / •	•	• / •	•	•	•	•
STM32F429[2]	180	512 K to 2 M	256	• / •	•	•	•	•	•	
STM32F427[2]	180	1 to 2 M	256	• / •	•	•	•	•		
Foundation lines										
STM32F446	180	256 K to 512 K	128	/ •	•	• / •	• / •			
STM32F407[2]	168	512 K to 1 M	192	• / •	•					
STM32F405[2]	168	512 K to 1 M	192	/ •						

Product lines	FCPU (MHz)	Flash (Kbytes)	RAM (KB)	RUN current (µA/MHz)	STOP current (µA)	Small package (mm)	FSMC (NOR/PSRAM/LCD support	QSPI	DFSDM	CAN 2.0B	DAC	TRNG	DMA Batch Acquisition Mode	USB 2.0 OTG FS
Access lines														
STM32F401	84	128 to 512	up to 96	Down to 128	Down to 10	Down to 3x3								•
STM32F410	100	64 to 128	32	Down to 89	Down to 6	Down to 2.553x 2.579					•	•	BAM	-
STM32F411	100	256 to 512	128	Down to 100	Down to 12	Down to 3.034x 3.22							BAM	•
STM32F412	100	512 to 1024	256	Down to 112	Down to 18	Down to 3.653x 3.651	•	•	•	•		•	BAM	• +LPM[4]
STM32F413[2]	100	1024 to 1536	320	Down to 115	Down to 18	Down to 3.951x 4.039	•	•	•	•		•	BAM+	• +LPM[4]

STM32F401

STM32F4xx Access Line: entry-level microcontrollers

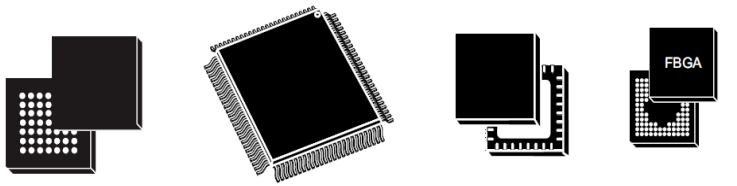
- ART Accelerator™
- SDIO
- USART, SPI, I²C
- I²S + audio PLL
- 16 and 32-bit timers
- 12-bit ADC (0.41 µs)
- True Random Number Generator
- Batch Acquisition Mode
- Low voltage 1.7 to 3.6 V
- Temperature:
- -40 °C to 125 °C

Access lines	FCPU (MHz)	Flash (Kbytes)	RAM (KB)	RUN current (µA/MHz)	STOP current (µA)	Small package (mm)	FSMC (NOR/ PSRAM/LCD support	QSPI	DFSDM	CAN 2.0B	DAC	TRNG	DMA Batch Acquisition Mode	USB 2.0 OTG FS
STM32F401	84	128 to 512	up to 96	Down to 128	Down to 10	Down to 3x3								•
STM32F410	100	64 to 128	32	Down to 89	Down to 6	Down to 2.553x 2.579					•	•	BAM	-
STM32F411	100	256 to 512	128	Down to 100	Down to 12	Down to 3.034x 3.22							BAM	•
STM32F412	100	512 to 1024	256	Down to 112	Down to 18	Down to 3.653x 3.651	•	•	•	•		•	BAM	• +LPM[1]
STM32F413[2]	100	1024 to 1536	320	Down to 115	Down to 18	Down to 3.951x 4.039	•	•	•	•		•	BAM+	• +LPM[1]

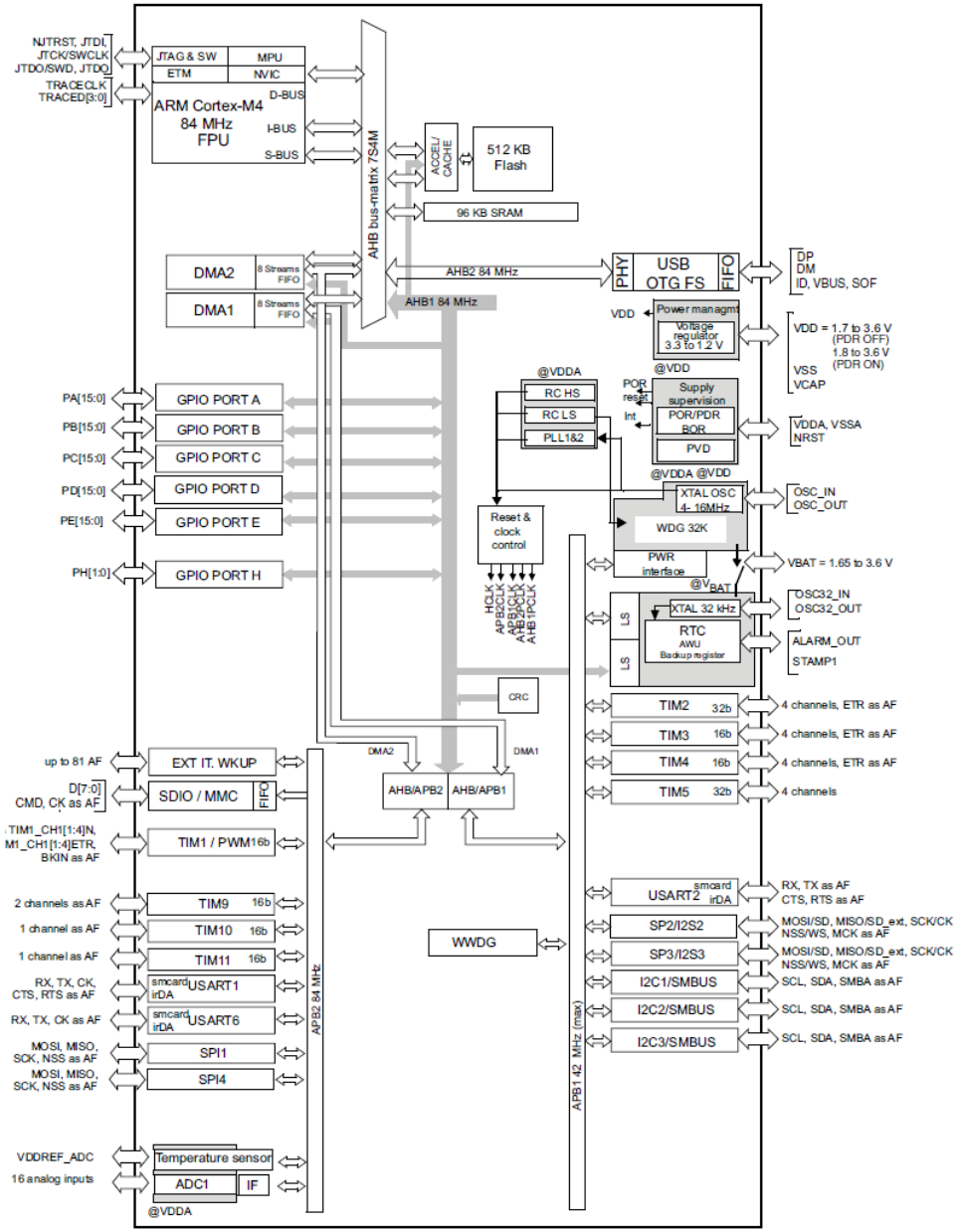
STM32F401RE

Flash size / RAM size (bytes)

	48-pin/49-pin UFQFPN/WLCSP	64-pin LQFP	100-pin LQFP/UFBGA
512 K / 96 K	STM32F401CE	STM32F401RE	STM32F401VE
384 K / 96 K	STM32F401CD	STM32F401RD	STM32F401VD
256 K / 64 K	STM32F401CC	STM32F401RC	STM32F401VC
128 K / 64 K	STM32F401CB	STM32F401RB	STM32F401VB



WLCSP49 (3.06 x 3.06 mm) LQFP100 (14 × 14 mm LQFP64 (10 × 10 mm) UFQFPN48 (7 × 7 mm) UFBGA100 (7 × 7 mm)



Clocks

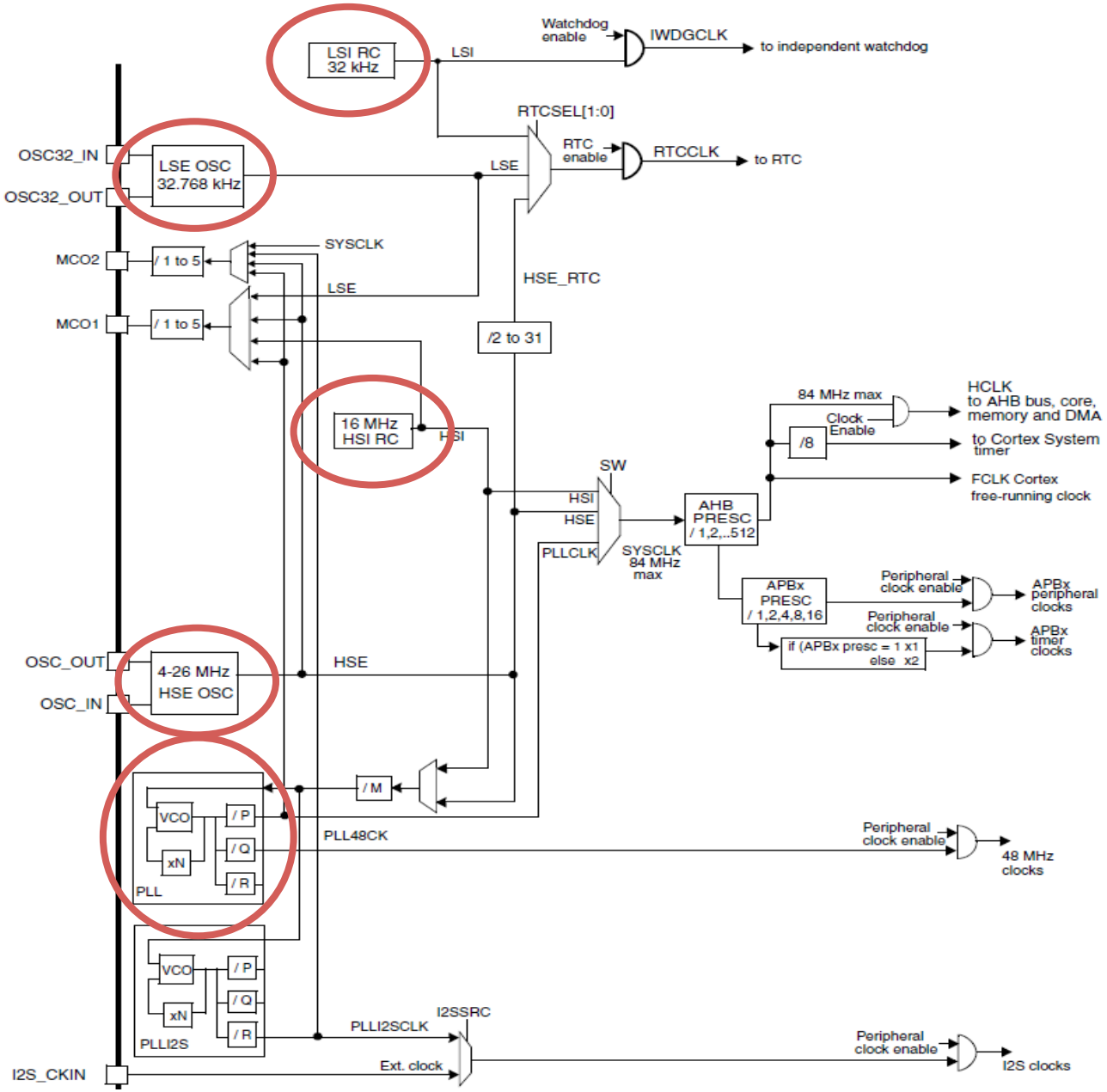
3 clock sources for system clock (SYSCLK):

- HSI oscillator clock
- HSE oscillator clock
- Main PLL (Phase-Locked Loop) clock.

2 secondary clock sources:

- a 32 kHz low-speed internal RC (LSI RC)
- a 32.768 kHz low-speed external crystal (LSE crystal)

Each source can be switched on/off independently when not used



Timers

Timer modules include:

- reference clock (internal or external)
- prescaler (reduce the clock frequency)
- counter (up, down, up/down mode)
- comparison registers

Timer modules are used for:

- generating interrupts
- generating pulse trains
- driving Pulse Width Modulation (PWM) outputs
- measuring pulse length of input signals
- counting events

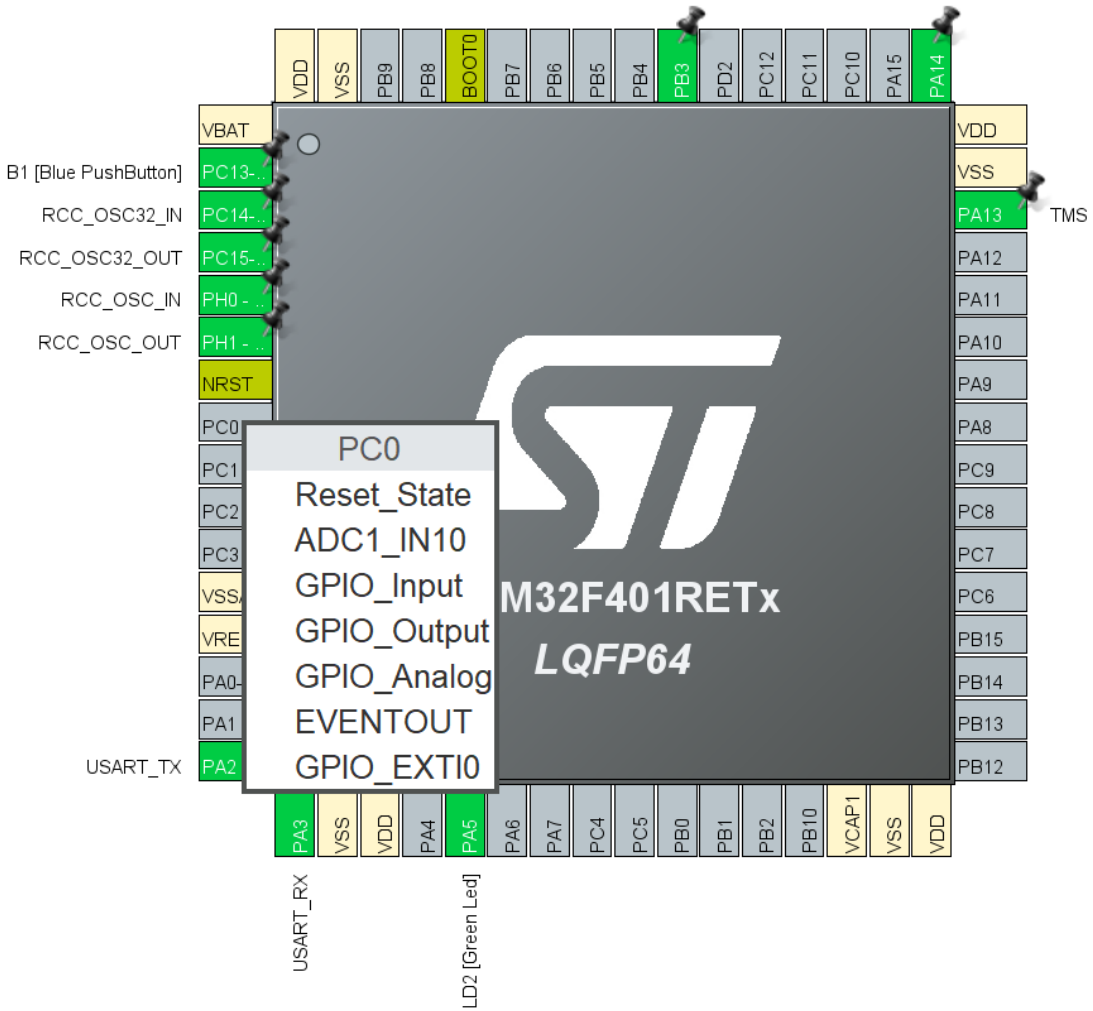
GPIOs

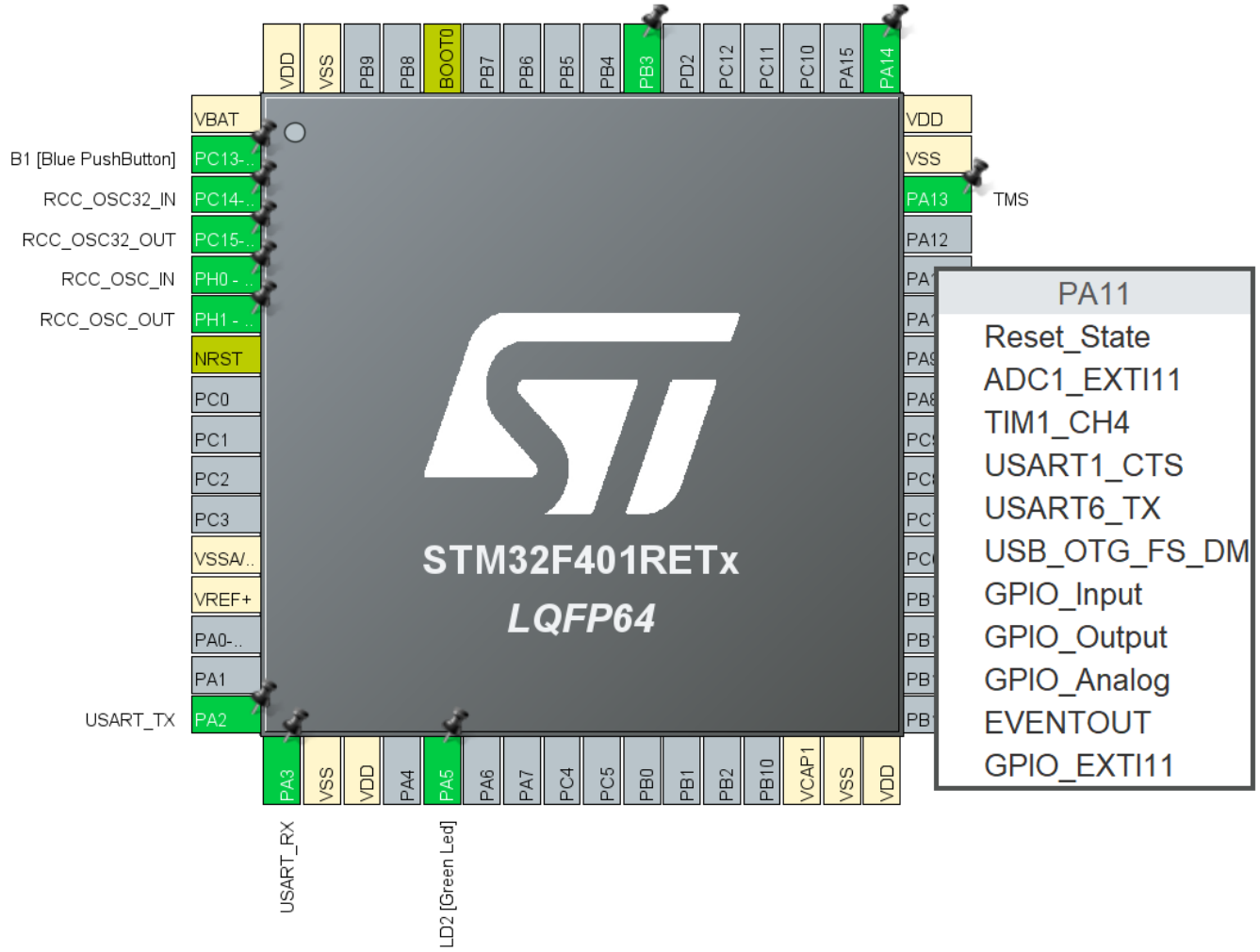
A GPIO is a **configurable pin** used to read or write digital signals.

Modes:

- **Input** → read external analog or digital signals (e.g., buttons, analog voltages…)
- **Output** → drive analog or digital signals (e.g., LEDs, analog reference voltages…)
- **Alternate Function** → used by peripherals (e.g., UART, SPI, Timers, etc.)

Graphical User Interface (GUI)





ADC

Analog to Digital Converters (ADC):

- 12-bit ADC
- 19 multiplexed channels

Can be triggered by timers.

Can be used in interrupt (IT) mode and DMA mode.

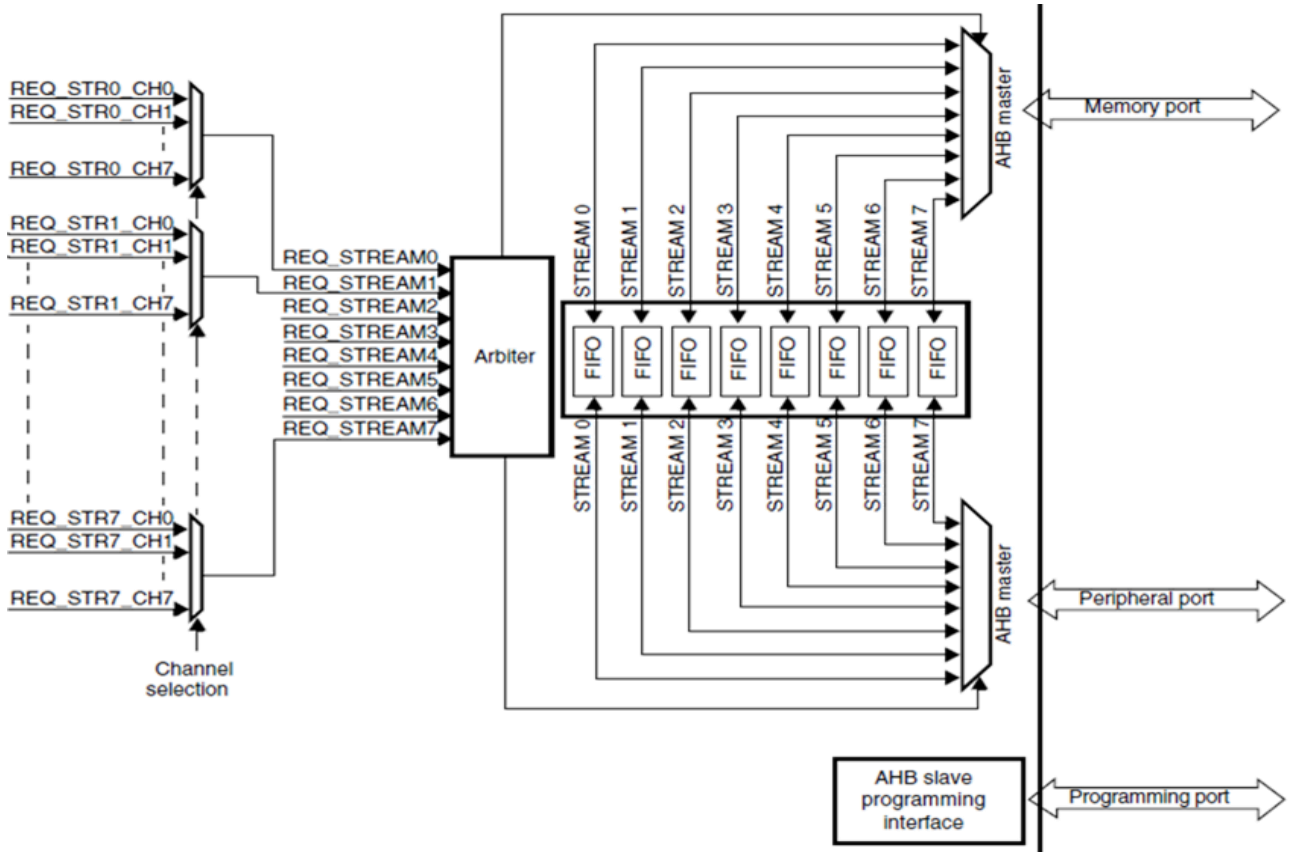
Direct Memory Access (DMA)

Hardware unit for **high speed data transfer**:

- peripherals ↔ memory
- memory ↔ memory

without the intervention of the **CPU**

→ CPU can continue to process the instructions without interruptions



Communication

I2C: 2 wire synchronous communication (SCK, SDA)

- 3 I2C bus interfaces can operate in multi-master and slave modes.
- Can support the standard (up to 100 kHz) and fast (up to 400 kHz) modes.

SPI: 4 wire synchronous full-duplex communication

- 4 SPI interfaces

USART: asynchronous single-wire half-duplex communication

- 3 USART interfaces

STM32 programming

HAL Hardware Abstraction Layer drivers:

- ✓ Hide the MCU and peripheral complexity
- ✓ Maximize software portability across different microcontrollers
- ✓ Exist for all peripherals
- × Larger overhead → performance penalty

LL Low-Level drivers:

- ✓ Better optimization → improved performance
- × Less portability
- × Require deep knowledge of the MCU and peripheral specifications

Both can be found in the **“Description of STM32F4 HAL and LL drivers” user manual**.

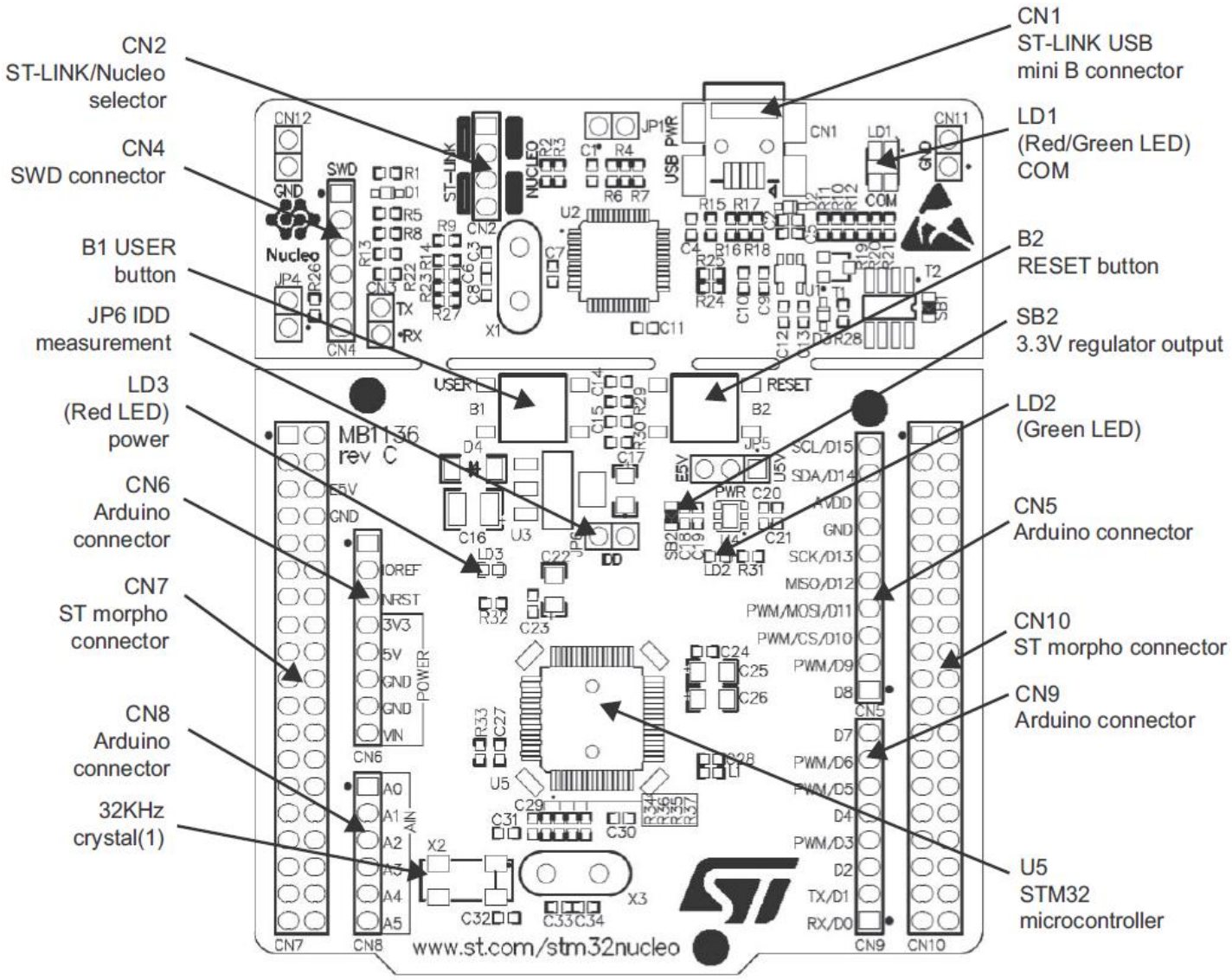
3. NUCLEO board

03

NUCLEO board features

- Connector for Arduino Uno Revision 3
- General purpose connector (ST morpho extension) male pin headers accessible on both side of the board full access to STM32 I/Os
- 3 LEDs:
  - LD1 (COM) for USB communication (green, red)
  - LD2 available for user (connected to PA5) (green)
  - LD3 MCU powered, +5V power is available (red)
- 2 pushbuttons:
  - B1 for USER connected to PC13
  - B2 for RESET
- LSE (low-speed external) oscillator at 32.768 kHz
- 3 interfaces supported by the USB (Virtual Com port, Mass storage, Debug port)

NUCLEO board features



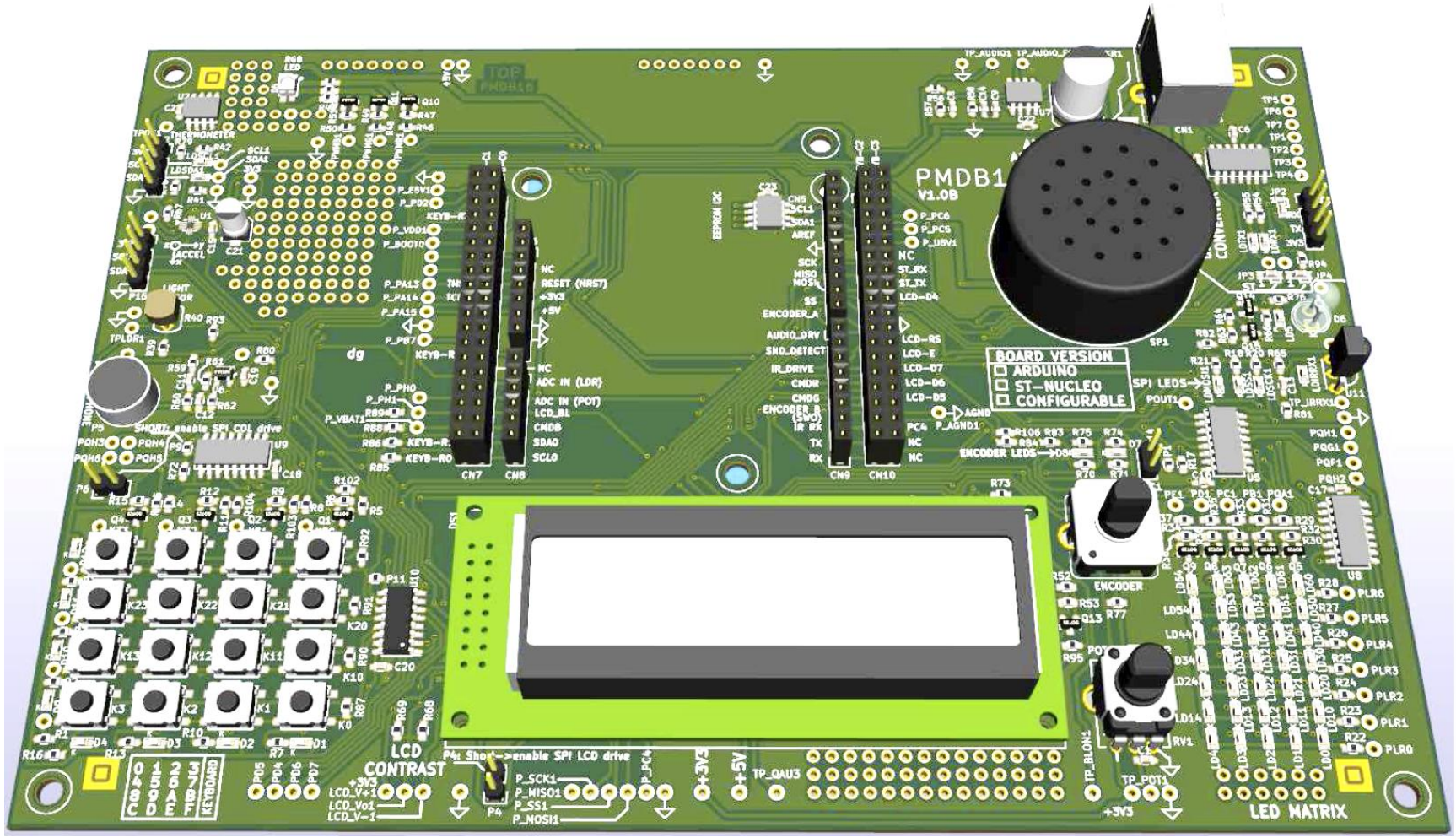
Information about the NUCLEO board can be found in:

- The NUCLEO board **datasheet**
- The NUCLEO board **user manual**

4. POLIMI expansion board

04

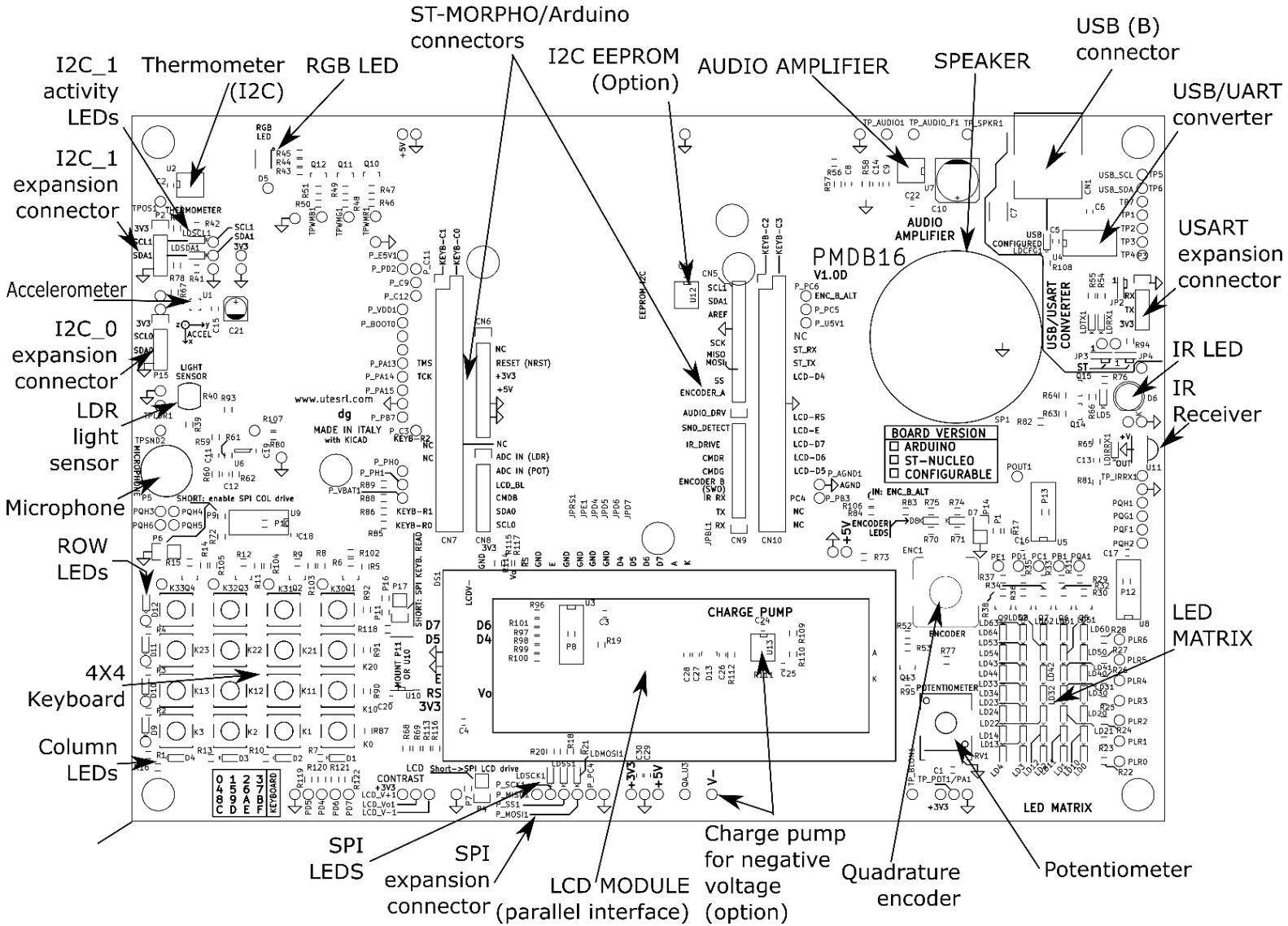
POLIMI expansion board



POLIMI board features

- standard 16x2 LCD alphanumeric module
- 5x7 LEDs array area, driven by two shift registers connected to the SPI interface
- 4x4 keypad, connected directly to the GPIOs
- incremental encoder
- microphone and comparator
- amplifier and speaker, connected to PWM
- RGB led, connected to three PWM-configurable outputs
- light sensor connected to ADC
- potentiometer connected to ADC
- thermometer, which communicates through I2C peripheral
- accelerometer, which communicates through I2C peripheral
- IR led and IR receiver

POLIMI board layout



How do we find to which MCU pin something is connected?

We need to use two schematics:

1. NUCLEO schematic
2. POLIMI board schematic

We will need to identify the sensor we are interested in on the POLIMI board schematic and to find where it is connected to the NUCLEO board.

Then, by looking at the NUCLEO schematic, we will identify the STM32F401RE pin we need to use in our code.

Try to find:

- The Blue Button pin (mounted directly on the NUCLEO)
- The microphone pin (mounted on the POLIMI board)



POLITECNICO MILANO 1863 | DEPARTMENT OF ELECTRONICS INFORMATION AND BIOENGINEERING

Arianna.devecchi@polimi.it
Federica.villa@polimi.it
www.polimi.it

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA